

实验六：跨平台综合项目实战

实验项目基本信息

- **实验编号**：d20301035106、d20301009206
- **学时分配**：6 学时
- **实验类型**：综合型
- **每组人数**：6 人
- **对应课程目标**：课程目标 1、2、3、4

🎯 核心步骤列表

📖 实验概览

1. **需求分析阶段**：团队协作完成项目需求文档与技术选型方案
2. **AI 辅助编码阶段**：各成员使用 TRAE 辅助各技术栈端的代码开发
3. **测试验证阶段**：制定测试计划，进行功能测试与跨端兼容性测试
4. **部署运维阶段**：完成多端应用打包部署，撰写技术选型对比报告与项目总结

📋 开发任务清单

任务阶段	主要内容	关键技术点
项目规划	需求分析、技术选型	团队协作、方案设计
多人开发	分工协作、Git 管理	分支管理、代码合并
多端实现	6 个技术栈开发	Android/iOS/Flutter/RN/Uniapp/MAUI
测试上线	功能测试、跨端测试	测试用例、问题修复
报告撰写	技术总结、选型分析	技术对比、分析报告

✅ 成功标准

- [] 6 个技术栈应用均可运行
- [] Git 团队协作流程规范
- [] AI 辅助开发流程体验良好
- [] 技术选型对比报告完整
- [] 项目总结报告完成

实验任务与案例

核心任务

团队开发某业务应用（建议选题涉及传感器或硬件交互场景，如运动健康、智能家居控制等），每人负责一个技术栈（Android/Flutter/React Native/Uniapp/MAUI/微信小程序），使用 Git 进行版本控制与协作，借助 AI 编程工具辅助代码生成与调试。

实战案例

团队开发“智慧健康”运动监测应用，支持：

- 步数统计（加速度计）
- 心率监测（模拟）

- 运动记录
 - 数据同步
- 每人负责一个技术栈实现，组内进行技术分享与对比分析。

第一课时：项目规划与需求分析（1 学时）

6.1 需求分析阶段

步骤 1：确定项目选题

1. ****可选项目****
 - 智慧健康（运动监测）
 - 智慧家居（设备控制）
 - 智慧校园（校园服务）
2. ****确定功能需求****
 - 用户登录
 - 数据展示
 - 传感器数据采集
 - 本地存储

步骤 2：技术选型方案

```

团队分工（6 人）：

- 成员 1：Android 原生开发（Kotlin）
- 成员 2：Flutter 开发（Dart）
- 成员 3：React Native 开发（JavaScript）
- 成员 4：Uniapp 开发（Vue）
- 成员 5：MAUI 开发（C#）
- 成员 6：微信小程序开发

```

步骤 3：编写需求文档

```markdown

# 智慧健康应用需求文档

### ## 1. 功能需求

- 用户登录
- 步数统计展示
- 运动记录列表
- 数据本地存储

### ## 2. 技术要求

- 各技术栈独立实现
- Git 协作开发
- AI 辅助编码

```

第二课时：Git 协作与开发环境准备（1 学时）

6.2 Git 团队协作配置

步骤 1：创建 Git 仓库

```bash

# 创建项目仓库

git init smart-health

cd smart-health

# 创建开发分支

git checkout -b develop

# 创建功能分支

git checkout -b feature/android

git checkout -b feature/flutter

git checkout -b feature/react-native

git checkout -b feature/uniapp

git checkout -b feature/maui

git checkout -b feature/wechat

```

步骤 2：编写.gitignore

```

# 各平台忽略文件

node\_modules/

.gradle/

build/

\*.apk

\*.ipa

dist/

```

6.3 项目初始化

步骤 1：各技术栈项目创建

```bash

# Android

android-studio -> SmartHealth

# Flutter

flutter create smart\_health\_flutter

```
React Native
npx react-native-init SmartHealthRN
```

```
Uniapp
HBuilderX -> 新建 uni-app
```

```
MAUI
Visual Studio -> 新建 MAUI 项目
```

```
微信小程序
微信开发者工具 -> 新建项目
```
```

第三四课时：多端开发实现（3 学时）

6.4 Android 开发

步骤 1：AI 辅助开发

1. 使用 TRAE 描述需求：Android Kotlin 计步器应用，显示步数列表
2. 生成基础代码框架
3. 完善业务逻辑

步骤 2：核心功能实现

```
```kotlin
// MainActivity.kt - 计步器核心
class MainActivity : AppCompatActivity() {

 private lateinit var tvStepCount: TextView
 private var stepCount = 0

 override fun onCreate(savedInstanceState: Bundle?) {
 super.onCreate(savedInstanceState)
 setContentView(R.layout.activity_main)

 tvStepCount = findViewById(R.id.tv_step_count)

 // 模拟步数更新
 Timer().schedule(object : TimerTask() {
 override fun run() {
 runOnUiThread {
 stepCount += Math.random() * 10
 tvStepCount.text = "步数: ${stepCount.toInt()}"
 }
 }
 }, 0, 1000)
```

```
 }
 }
 ...
}
```

### ### 6.5 Flutter 开发

#### #### 步骤 1: AI 辅助开发

1. 使用 TRAE 描述需求: Flutter 计步器应用, Provider 状态管理
2. 生成代码框架

#### #### 步骤 2: 核心功能实现

```
```dart  
// main.dart  
class StepCounter extends ChangeNotifier {  
  int _steps = 0;  
  int get steps => _steps;  
  
  void increment() {  
    _steps++;  
    notifyListeners();  
  }  
}  
  
class HomePage extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  
      appBar: AppBar(title: Text('智慧健康')),  
      body: Consumer<StepCounter>(  
        builder: (context, counter, child) {  
          return Center(  
            child: Text('步数: ${counter.steps}'),  
          );  
        },  
      ),  
    );  
  }  
}  
```
```

### ### 6.6 React Native 开发

```
```jsx  
// App.js  
const App = () => {  
  const [steps, setSteps] = useState(0);  
}
```

```

useEffect(() => {
  const interval = setInterval(() => {
    setSteps(s => s + Math.floor(Math.random() * 10));
  }, 1000);
  return () => clearInterval(interval);
}, []);

return (
  <View style={styles.container}>
    <Text style={styles.text}>步数: {steps}</Text>
  </View>
);
};
```

```

### ### 6.7 Uniapp 开发

```

```vue
<!-- pages/index/index.vue -->
<template>
  <view class="container">
    <text class="title">智慧健康</text>
    <text class="steps">步数: {{steps}}</text>
  </view>
</template>

<script>
export default {
  data() {
    return { steps: 0 }
  },
  onLoad() {
    setInterval(() => {
      this.steps += Math.floor(Math.random() * 10);
    }, 1000);
  }
}
</script>
```

```

### ### 6.8 MAUI 开发

```

```csharp
// MainPage.xaml.cs
public partial class MainPage : ContentPage
{
  public MainPage()

```

```

    {
        InitializeComponent();

        Device.StartTimer(TimeSpan.FromSeconds(1), () =>
        {
            Steps += Random.Shared.Next(10);
            return true;
        });
    }

    public static readonly BindableProperty StepsProperty =
        BindableProperty.Create(nameof(Steps), typeof(int), typeof(MainPage), 0);

    public int Steps
    {
        get => (int)GetValue(StepsProperty);
        set => SetValue(StepsProperty, value);
    }
}
```

```

### ### 6.9 微信小程序开发

```

```javascript
// pages/index/index.js
Page({
    data: {
        steps: 0
    },

    onLoad() {
        setInterval(() => {
            this.setData({
                steps: this.data.steps + Math.floor(Math.random() * 10)
            });
        }, 1000);
    }
});
```

```

```

```xml
<!-- pages/index/index.wxml -->
<view class="container">
    <text>智慧健康</text>
    <text>步数: {{steps}}</text>
</view>
```

```

----

## ## 第五课时：测试验证与打包部署（1 学时）

### ### 6.10 测试验证阶段

#### #### 步骤 1：功能测试

1. 编写测试用例
2. 执行功能测试
3. 记录测试结果

```markdown

测试用例

功能测试

- [] 应用启动正常
- [] 步数统计显示
- [] 数据更新正常
- [] 页面切换流畅

兼容性测试

- [] 各平台运行正常
- [] 界面显示正确
- [] 性能表现良好

```

#### #### 步骤 2：问题修复

1. 汇总测试问题
2. 分析原因
3. 修复缺陷

### ### 6.11 打包部署

#### #### 步骤 1：各平台打包

```bash

Android

./gradlew assembleRelease

Flutter

flutter build apk --release

React Native

npx react-native build-android

MAUI

dotnet publish -f net8.0-android


```

- #### 步骤 2：体验版发布
- Android APK 安装测试
  - iOS IPA 打包（模拟器）
  - 小程序上传体验版

---

## ## 第六课时：技术选型对比报告（1 学时）

### ### 6.12 编写技术选型对比报告

#### #### 步骤 1：整理对比数据

```markdown

技术选型对比报告

1. 各技术栈开发效率对比

| 技术栈 | 开发用时 | 代码量 | 难度评分 |
|--------------|------|-----|------|
| Android | X 小时 | Y 行 | Z 分 |
| Flutter | X 小时 | Y 行 | Z 分 |
| React Native | X 小时 | Y 行 | Z 分 |
| Uniapp | X 小时 | Y 行 | Z 分 |
| MAUI | X 小时 | Y 行 | Z 分 |
| 微信小程序 | X 小时 | Y 行 | Z 分 |

2. 性能对比

- 启动速度
- 内存占用
- 流畅度

3. 适用场景分析

4. 技术选型建议

```

### ### 6.13 项目总结报告

```markdown

项目总结报告

1. 项目概述

- 项目名称
- 开发目标
- 团队成员

2. 开发过程

- 需求分析
- 技术选型
- 开发实现
- 测试上线

3. 收获与体会

- 技术成长
- 团队协作
- AI 工具使用

4. 问题与改进

- 遇到的问题
 - 解决方案
 - 改进方向
- ````

6.14 组内技术分享

步骤 1: 各成员分享

1. 技术实现方案
2. 遇到的问题
3. 解决方案
4. 心得体会

步骤 2: 讨论总结

- 各技术栈优劣
- 技术选型建议
- 未来学习方向

总结与思考

实验总结

- 掌握多技术栈开发能力
- 体验 Git 团队协作流程
- 学会 AI 辅助开发
- 完成技术选型分析

课后思考

1. 如何选择合适的技术栈
2. AI 对开发工作的影响
3. 未来移动开发趋势